

# Lab - 1 Software Engineering

Name: Bhuvi Bhardwaj

SRN: PES2UG24AM041

## Overview

This is a developer productivity tool: you feed it an OpenAPI 3.0 spec (YAML or JSON), and it spins up mock REST endpoints that validate against that schema and can simulate realistic response latency. It's built for two kinds of people — the API Developer building the spec, and the QA Engineer who needs something reliable to test against before the real backend exists.

## 1. Requirements Specification

| Req ID | Type       | Description                                                                                                                                                              | Priority | Acceptance Criteria                                                                                                                                                                          | Rationale                                                                                                        |
|--------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| FR-001 | Functional | The system shall parse OpenAPI 3.0 schema definitions and generate dynamic HTTP mock endpoints that return randomized, schema-compliant JSON payloads.                   | High     | Pass: hitting a mock endpoint returns JSON that matches the field types in the spec.Fail: an endpoint path that isn't in the spec still returns 200 OK.                                      | This is the whole point of the tool — if it can't turn a spec into working mock endpoints, nothing else matters. |
| FR-002 | Functional | The system shall check an uploaded OpenAPI 3.0 YAML or JSON file against the OpenAPI schema before using it, and reject it with a clear error message if it's malformed. | High     | Pass: uploading a spec that's missing a required field returns an error naming the field and its location.Fail: a broken spec gets accepted and mock endpoints are generated from it anyway. | Nobody wants to debug weird mock behaviour for twenty minutes only to discover the spec itself was invalid.      |

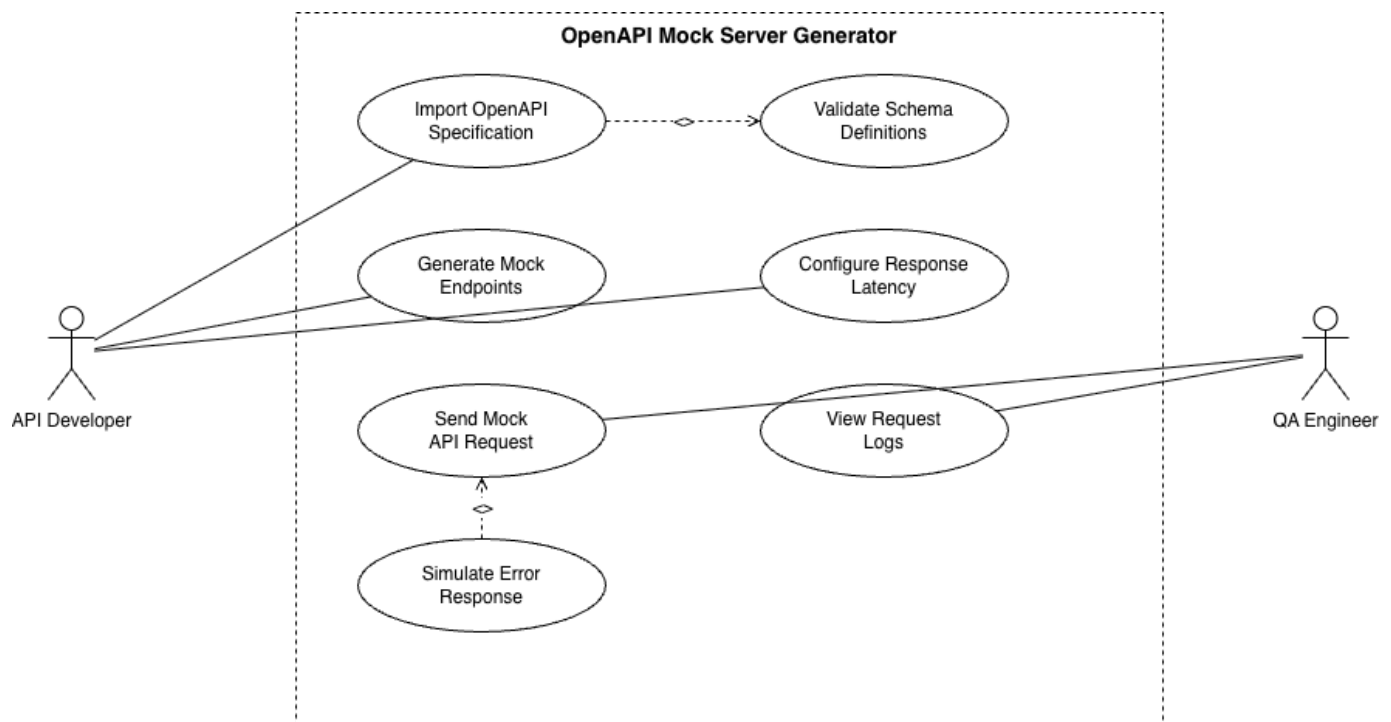
| Req ID | Type       | Description                                                                                                                                                                    | Priority | Acceptance Criteria                                                                                                                                                                                                           | Rationale                                                                                                               |
|--------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| FR-003 | Functional | The system shall let an API Developer set a simulated response delay — either a fixed value or a random range — for each mock endpoint.                                        | Medium   | Pass: an endpoint set to 200–400 ms responds inside that window on 20 out of 20 test calls. Fail: changing the latency setting makes no real difference to response time.                                                     | QA needs a way to reproduce slow-network conditions to test how the client app handles timeouts and retries.            |
| FR-004 | Functional | The system shall let a QA Engineer save a custom example response for a specific endpoint and status code, overriding the normal randomized output.                            | Medium   | Pass: after saving a custom payload for GET /users/{id} → 200, the next call to that endpoint returns exactly that payload. Fail: the endpoint keeps returning randomly generated data even after a custom example was saved. | QA often needs to test one specific tricky case — an empty list, a weird edge value — without touching the spec itself. |
| FR-005 | Functional | The system shall record every mock request and response — timestamp, method, path, status code, latency — and make that log available through a dashboard or a query endpoint. | Medium   | Pass: querying the log for a given path shows every matching request from the current session with the right timestamps. Fail: requests made to a mock endpoint simply don't show up in the log.                              | When a test fails, the first question is always "what actually got called?" — the log answers that.                     |

## Non-Functional Requirements

| Req ID  | Type          | Description                                                                                                                                                  | Priority | Acceptance Criteria                                                                               | Rationale                                                                                         |
|---------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| NFR-001 | Nonfunctional | The mock server engine must be able to handle 1,000 mock API requests per second while keeping the configured simulated latency accurate within $\pm 10$ ms. | High     | Pass: load-testing confirms the target throughput and latency accuracy under peak simulated load. | Mock servers get dropped into load-testing pipelines all the time — if the mock layer is the slow |

| Req ID  | Type          | Description                                                                                                                                                                          | Priority | Acceptance Criteria                                                                                                                                                                                        | Rationale                                                                                     |
|---------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|         |               |                                                                                                                                                                                      |          |                                                                                                                                                                                                            | part, it defeats the purpose.                                                                 |
| NFR-002 | Nonfunctional | The system shall be easy enough to get running that a first-time user can import a spec and get a working mock endpoint within 5 minutes, without installing anything beyond Docker. | Medium   | Pass: a first-time user, following only the README, gets a working mock endpoint from a sample spec in under 5 minutes.Fail: setup needs undocumented steps or takes longer than 5 minutes in a timed run. | A tool that's a pain to set up just doesn't get adopted, no matter how good the features are. |

## UML Use-Case Diagram:



## Actor

API Developer

## Preconditions

The API Developer is logged into the mock server tool and has an OpenAPI 3.0 spec file (YAML or JSON) ready on their machine.

## Postconditions

A valid spec has been stored, its schema has been validated, and a full set of mock endpoints is live and ready to receive requests.

## Main Success Scenario

1. The API Developer selects “Import Specification” and chooses a YAML or JSON file from their machine.
2. The system uploads the file and reads its contents.
3. The system runs the include use case “Validate Schema Definitions”, checking the file against the OpenAPI 3.0 schema.
4. The system confirms the spec is valid and shows the developer a summary of the endpoints it found.
5. The system generates a mock HTTP endpoint for every path and method defined in the spec.
6. The system displays the list of live mock endpoints, along with their base URL, to the developer.
7. The use case ends successfully.

## Alternate Flow — 3a. Schema Validation Fails

- 3a1. The system detects that the uploaded file doesn't conform to the OpenAPI 3.0 schema.
- 3a2. The system rejects the import and shows an error message naming the specific field and location of the problem.
- 3a3. The developer fixes the spec file and re-submits it, returning the flow to step 1.
- 3a4. If the developer cancels instead of retrying, the import is abandoned and no mock endpoints are created.